ITAI 3377 – AI at the Edge & IIOT Env.

Prof. Sitaram Ayyagari

Natalia Solorzano W207818526

Spring 2026 – 17181

L03: Deploying a Simple AI Model on a Simulated Edge Device using Visual Studio Code

A. Introduction

The purpose of this lab was to explore the fundamentals of Edge AI by training a lightweight machine learning model and preparing it for deployment on edge devices. Instad of relying on clould-based inference, Edge AI focuses on performing model inference directly on local hardware, which reduces latency, improves privacy, and enables real-time decision making constrained environments.

In this lab, a Convolutational Neural Network (CNN) was trained using the MNIST handwritten digit dataset. After training, the model converted into the TensorFlow Lite (TFLite) format to simulate deployment in an edge computing scenario.

B. Dataset Description

The MNIST dataset consists of grayscale images of handwritten digits ranging from 0 to 9. Each image is 28x28 pixels in size and labeled with its corresponding digit. MNIST is widely used as an introductory dataset for image classification tasks due to its simplicity and standardized format.

For this lab:

-The dataset was loaded directly through TensorFlow/Keras utilities.

-Images were normalized to improve training performance

-Labels were used for supervised learning

C. Model Architecture

A Convolutional Neural Network (CNN) was implemented to classify the handwritten digits. CNNs are particulary effective for image-based tasks because they automatically learn spatial features such as edges, shapes, and patterns.

The model architecture included:

-An input layer for 28x28 grayscale images

-Convolutional layers for feature extraction

-Pooling layers to reduce spatial dimensions

-A fully connected (dense) layer for classification

-A softmax output layer with 10 classes (digits 0-9)

The model was complied using categorical cross-entropy loss and the Adam optimizer.

D. Training Process

The model was trained for three epochs using the training portion of the MNIST dataset. During training, both accuracy and loss were monitored to evaluate learning progress.

Key Observations

-Accuracy incresed rapidly during the first epoch

-Loss steadily decreased across all epochs

-Validation accuracy remained high, indicating good generalization

By the final epoch, the model achieved approximately 98% training accuracy and 98% validation accuracy, demonstrating effective learning.

E. Edge Deployment Preparation

After training, the model was saved and converted to TensorFlow Lite (.tflite) format. TensorFlow Lite is designed for deployment on resource-constrained edge devices such as microcontrollers, embedded systems, and mobile devices.

This conversion step is essential for Edge AI because it:

-Reduces model size

-Improves inference speed

-Enables deployment without continous clould connectivity

The successful generation of the model.tflite file confirms that the trained model is suitable for edge-based inference.

F. Results

The final trained model demonstrated strong performance:

-Training Accuracy: ~98.3%

-Validation Accuracy: ~98.0%

-Low validation loss, indicating minimal overfitting

Additionally, the training output confirmed that the model was successfully saved and prepared for edge deployment.

A screenshot of the terminal output and project directory structure was captured to verify successful training and TensorFlow Lite model generation.

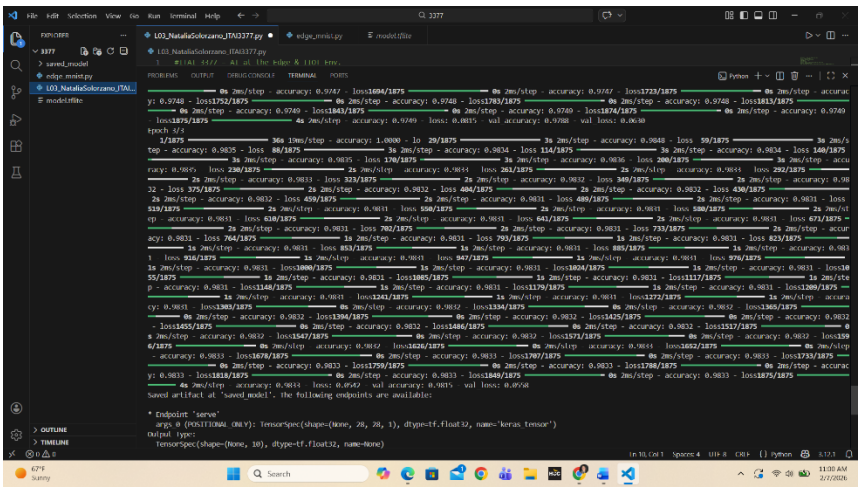
G. Conclusion

This lab provded hands-on experience with the Edge AI workflow, from dataset loading and model training to edge deployment preparation. Training a CNN on the MNIST dataset and converting it into TensorFlow Lite model demonstrated how AI models can be optimized for real-world environments.

Edge AI plays a critical role in modern IIoT systems by enabling real-time intelligence, reducing latency, and minimizing reliance on cloud infrastructure. This experiments illustrates how even simple models can be adapted for edge computing applications.

H. Screenshot Evidence

Figure 1 shows the terminal output displaying successful training across three epochs and confirmation of the generated model.tflite file, verifying readiness for edge deployment.



I. Tools and Technologies Used

-Python 3.12

-TensorFlow/Keras

-TensorFlowLite

-Visual Studio Code

-MNIST Dataset

References

TensorFlow Documentation. (n.d.). https://www.tensorflow.org/